

HormuzNet: Monitoramento Marítimo Descentralizado e Resiliência em Sistemas Críticos

Luis Felipe Pereira de Carvalho¹

¹Departamento de Tecnologia – Universidade Estadual de Feira de Santana (UEFS)
44036-900 – Feira de Santana – Bahia

luisoftbr@outlook.com.br

Abstract. *[Context] Maritime monitoring of strategic routes demands distributed architectures capable of integrating heterogeneous sensors with high availability and no single point of failure. [Problem] Centralized architectures introduce a critical single point of failure: if the central server becomes inoperative, the entire monitoring and maritime rescue infrastructure is immediately disrupted. [Contribution] This work presents HormuzNet, a decentralized maritime monitoring system for the Strait of Hormuz that replaces the centralized model with an autonomous cooperative mesh of brokers, using Lamport's Logical Clocks for global event ordering and distributed mutual exclusion. [Methodology] The system was implemented in Go using native UDP Multicast for redundant telemetry, persistent TCP with fallback logic for drone coordination, and WebSocket for real-time tactical visualization. The infrastructure was validated in a containerized environment with 34 Docker containers distributed across two physical hosts. [Results] The architecture demonstrated effective elimination of single points of failure, automatic broker recovery, consistent mission dispatching with an average latency under 1.5 seconds, and correct Lamport clock synchronization under simultaneous load from 18 sensors.*

Resumo. *[Contexto] O monitoramento marítimo de rotas estratégicas exige arquiteturas distribuídas capazes de integrar sensores heterogêneos com alta disponibilidade e ausência de pontos únicos de falha. [Problema] Arquiteturas centralizadas introduzem um ponto único de falha crítico: se o servidor central torna-se inoperante, toda a infraestrutura de monitoramento e salvamento marítimo é imediatamente interrompida. [Contribuição] Este trabalho apresenta o HormuzNet, um sistema descentralizado de monitoramento marítimo para o Estreito de Ormuz que substitui o modelo centralizado por uma malha cooperativa autônoma de brokers, utilizando Relógios Lógicos de Lamport para ordenação global de eventos e exclusão mútua distribuída. [Metodologia] O sistema foi implementado em Go com UDP Multicast nativo para telemetria redundante, TCP persistente com lógica de fallback para coordenação de drones e WebSocket para visualização tática em tempo real. A infraestrutura foi validada em ambiente containerizado com 34 contêineres Docker distribuídos em duas máquinas físicas. [Resultados] A arquitetura demonstrou eliminação efetiva de pontos únicos de falha, recuperação automática de brokers, despacho consistente de missões com latência média inferior a 1,5 segundos e sincronização correta dos relógios de Lamport mesmo sob carga simultânea de 18 sensores.*

1. Introdução

O monitoramento de infraestruturas críticas, como o Estreito de Ormuz, constitui um desafio complexo no campo dos sistemas distribuídos. O estreito é uma das rotas marítimas mais estratégicas do mundo, responsável por aproximadamente 20% do tráfego global de petróleo [Tanenbaum et al. 2021]. A necessidade de rastrear continuamente embarcações, detectar anomalias ambientais e despachar recursos de resposta a emergências exige uma arquitetura de software robusta, de alta disponibilidade e capaz de operar mesmo diante de falhas parciais na infraestrutura.

Sistemas baseados em modelo de *broker* centralizado, como o FarmNode desenvolvido no projeto anterior, introduzem um ponto único de falha (*Single Point of Failure* – SPOF) crítico: se o servidor central torna-se inoperante, toda a infraestrutura de monitoramento e salvamento marítimo é interrompida imediatamente. Adicionalmente, a alocação concorrente de recursos escassos — como drones de patrulhamento disputados por múltiplas ocorrências simultâneas — pode resultar em inanição de tarefas ou impasse (*deadlock*) distribuído.

Para mitigar esses problemas, este trabalho apresenta o **HormuzNet**, um sistema de monitoramento marítimo descentralizado baseado em uma malha cooperativa de *brokers*. O HormuzNet elimina SPOFs ao distribuir o processamento por meio de uma arquitetura híbrida que combina o modelo cliente-servidor em dois níveis com uma malha ponto a ponto (P2P). A consistência de dados e a ordenação global de eventos são garantidas pelos Relógios Lógicos de Lamport [Lamport 1978], enquanto a exclusão mútua distribuída resolve a concorrência pelo despacho de drones.

O sistema foi implementado em Go com protocolos nativos de rede: UDP/IP Multicast para telemetria redundante, TCP/IP persistente com lógica de *fallback* para coordenação de drones autônomos, e WebSocket para visualização tática em tempo real. A infraestrutura foi validada com 34 contêineres Docker distribuídos entre duas máquinas físicas interconectadas por rede LAN Gigabit.

Os resultados demonstram que a malha de *brokers* reage autonomamente a falhas por meio de *Ring Failover* e eleição Bully, realizando o despacho de drones com latência média inferior a 1,5 s e sem inconsistências nas filas de prioridade distribuídas.

2. Fundamentação Teórica

A resiliência do HormuzNet apoia-se em quatro pilares teóricos: arquitetura distribuída descentralizada, protocolos de rede em camadas, concorrência gerenciada e algoritmos de coordenação lógica.

2.1. Arquiteturas Distribuídas e Modelos de Comunicação

Sistemas distribuídos organizam nós autônomos que cooperam por troca de mensagens [Tanenbaum et al. 2021]. O modelo cliente-servidor tradicional centraliza o processamento em um único ponto, o que introduz pontos únicos de falha e gargalos de desempenho; a descentralização P2P distribui responsabilidades entre os nós para obter maior tolerância a falhas e redundância [Tanenbaum et al. 2021]. O HormuzNet combina os dois modelos: sensores e drones operam como clientes dos *brokers* de setor (camada 1), enquanto os *brokers* formam entre si uma malha P2P cooperativa (camada 2).

Os protocolos de rede foram selecionados com base no *trade-off* entre confiabilidade e latência. O UDP (RFC 768 [Postel 1980]) sobre IP (RFC 791 [Postel 1981]) oferece entrega sem conexão e baixo *overhead*, adequado para telemetria redundante via *Multicast* (RFC 1112 [Deering 1989]), em que a perda eventual de um datagrama é tolerável diante do fluxo contínuo dos sensores. O TCP (RFC 793 [Institute 1981]) fornece canal bidirecional confiável e com entrega ordenada, indispensável para mensagens de coordenação da malha (*gossip*, Lamport, Bully) e comandos de despacho de drones. O WebSocket (RFC 6455 [Fette and Melnikov 2011]) acrescenta comunicação *full-duplex* em tempo real sobre TCP, permitindo que o monitor tático receba atualizações instantâneas sem a necessidade de *polling* HTTP.

2.2. Concorrência em Go e Estruturas de Dados Distribuídas

A linguagem Go implementa o modelo CSP (*Communicating Sequential Processes*) [Donovan and Kernighan 2015]: *goroutines* são *threads* leves gerenciadas em espaço de usuário, enquanto canais proveem comunicação segura entre elas. Para acesso direto à memória compartilhada, *mutexes* refinados por recurso (*dronesMu*, *ocorrenciasMu*) protegem regiões críticas sem introduzir gargalos globais. Filas de prioridade baseadas em *heap* binário garantem que ocorrências críticas sejam atendidas primeiro; o algoritmo de *aging* previne inanição ao incrementar gradualmente a prioridade de elementos que aguardam há muito tempo na fila.

2.3. Relógio Lógico de Lamport e Algoritmo Bully

Lamport [Lamport 1978] propôs *timestamps* lógicos para ordenação total de eventos em sistemas distribuídos sem relógio físico global: cada processo mantém um contador inteiro incrementado a cada evento local e atualizado ao receber mensagens externas, preservando a relação de causalidade entre eventos. No HormuzNet, o relógio de Lamport é sincronizado a cada mensagem trocada na malha P2P, garantindo que ocorrências inseridas concorrentemente em *brokers* distintos sejam ordenadas de forma global e determinística.

Para a eleição do *broker* líder, utiliza-se o Algoritmo Bully [Tanenbaum et al. 2021]: ao detectar a ausência de *heartbeat* do coordenador, um nó envia *ELEICAO* a todos os *brokers* com identificador superior; o nó de maior ID que responder torna-se o novo líder e anuncia sua posição à malha, garantindo convergência rápida mesmo em cenários de falha em cascata.

2.4. Docker e Emulação Distribuída

A tecnologia de contêineres Docker isola processos compartilhando o *kernel* do sistema hospedeiro, proporcionando portabilidade e consistência de ambiente entre máquinas distintas. O Docker Compose permite definir topologias de múltiplos contêineres e suas redes virtuais em um único arquivo declarativo. No HormuzNet, essa tecnologia foi empregada para emular a topologia distribuída com 34 contêineres independentes em máquinas rodando Linux Ubuntu e Linux Mint.

3. Metodologia

Esta seção descreve os materiais, métodos e decisões de projeto adotados no desenvolvimento do HormuzNet. São detalhados a arquitetura do sistema, os protocolos de comunicação implementados, os algoritmos distribuídos de tolerância a falhas e o ambiente de emulação utilizado para validação.

3.1. Material e Método

O desenvolvimento e a avaliação experimental do HormuzNet basearam-se em computadores conectados via rede física Gigabit Ethernet no Laboratório LARSID da Universidade Estadual de Feira de Santana (UEFS). Os hosts executaram distribuições Linux (**Ubuntu** e **Mint**). O ambiente de software compreendeu:

- **Linguagem de Programação:** Go (versão 1.21.2);
- **Motor de Contêineres:** Docker (versão 24.0.5);
- **Orquestrador:** Docker Compose (versão 2.20.2).

Foram testadas duas topologias: a **local**, com todos os contêineres em um único *host* compartilhando rede virtual *bridge*, e a **distribuída**, com a infraestrutura dividida entre duas máquinas físicas via LAN Gigabit. O cenário padrão instanciou **34 contêineres**: 9 *brokers* (B1–B9), 6 drones autônomos, 18 sensores inteligentes (2 por setor) e 1 monitor tático. As métricas foram coletadas via chamadas *curl* ao endpoint `/api/estado`, logs em tempo real pelo script `terminais.sh` e saída do comando `docker stats`.

3.2. Arquitetura do Sistema

O HormuzNet afasta-se do modelo hierárquico centralizado do FarmNode, que dependia de um único *broker* como SPOF. Em vez disso, adota um modelo **cliente-servidor em dois níveis**:

- (i) No primeiro nível, sensores e drones comportam-se como clientes dos *brokers* de setor (servidores locais);
- (ii) No segundo nível, os *brokers* formam entre si uma malha descentralizada P2P cooperativa.

Essa topologia híbrida garante que mensagens de telemetria e coordenação circulem de forma redundante e resiliente pela malha geográfica. A Figura 1 apresenta a organização dos componentes e a dinâmica de auto-organização da malha.

3.3. Protocolo de Comunicação

A troca de informações ocorre por meio de canais específicos detalhados na Tabela 1.

Tabela 1. Portas, protocolos e formatos dos canais de comunicação do HormuzNet.
Fonte: Próprio Autor.

Fluxo	Protocolo	Porta	Formato	Propósito
Sensor → Broker	UDP/IP Multicast	9876	JSON UTF-8	Telemetria e alertas
Broker ↔ Broker	TCP/IP P2P	6000–6008	JSON/linha	<i>Gossip</i> , Lamport, Bully
Drone ↔ Broker	TCP/IP	6000–6008	JSON/linha	Registro e despacho
Broker → Monitor	TCP/IP	6000–6008	JSON/linha	Logs e descoberta
Monitor → Dashboard	WebSocket	8085	JSON <i>frame</i>	<i>Snapshot</i> tático

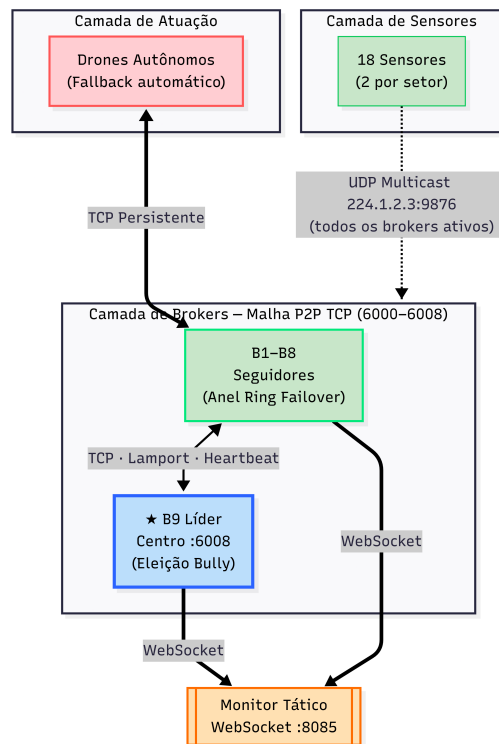


Figura 1. Arquitetura descentralizada do HormuzNet com camadas de sensores, brokers e drones. Fonte: Próprio Autor.

Na telemetria, sensores publicam datagramas JSON no grupo *multicast* 224.1.2.3:9876; a delimitação de mensagem é o próprio tamanho do datagrama UDP/IP [Postel 1980, Postel 1981]. Nos fluxos TCP/IP de coordenação, cada mensagem JSON é terminada por `\n` e lida via `bufio.Scanner`.

3.4. Descoberta, Filas e Concorrência

Ao iniciar, cada *broker* seguidor conecta-se ao líder e recebe a lista de membros ativos (PEER_LIST), estabelecendo suas conexões TCP P2P dinamicamente. Drones registram-se no *broker* de seu setor e, em caso de falha deste, executam *fallback* automático para o vizinho seguinte na lista de setores. Cada *broker* mantém um *heap* binário *thread-safe* com capacidade para até 262.144 ocorrências; o despacho seleciona o drone disponível de menor distância cartesiana ao incidente, emite o comando `DESPACHAR_DRONE` via TCP e propaga a remoção do evento por *broadcast* à malha. A concorrência é gerenciada com *goroutines* por *socket* e *mutexes* por recurso, com *timeout* de escrita de 2 s para prevenir bloqueios.

3.5. Confiabilidade: Heartbeat e Ring Failover

Brokers emitem *heartbeats* a cada 5 s para seus vizinhos P2P. Após 15 s sem resposta, o nó é marcado como inativo: o vizinho anterior no anel lógico assume o setor (*Ring Failover*), os drones do *broker* inativo reconectam-se por *backoff* ao próximo vizinho disponível e, caso o líder tenha falhado, os seguidores iniciam imediatamente o algoritmo Bully para eleição de novo coordenador.

3.6. Emulação e Testes

O ambiente de emulação injeta distúrbios realísticos de forma contínua: ocorrências permanecem na fila sob *aging* de 30 s, com incremento de criticidade a cada ciclo sem atendimento; drones em patrulha entram em alerta de ociosidade após 60 s sem missão; e cada drone possui 10% de probabilidade de ser abatido em trânsito, forçando o *broker* a registrar a perda e re-enfileirar a ocorrência na malha.

4. Resultados

A avaliação prática da arquitetura proposta sob condições de falha e estresse fornece evidências sobre sua viabilidade e eficiência. Esta seção apresenta os resultados funcionais e de desempenho obtidos e estabelece um comparativo com a arquitetura do FarmNode.

4.1. Ambiente de Avaliação

A validação experimental ocorreu por meio de cenários simulados de estresse e injeção de falhas no ambiente de contêineres descrito na Metodologia. A malha foi instanciada com 9 *brokers*, 6 drones autônomos e 18 sensores georreferenciados. A avaliação distribuída envolveu a partição física do processamento entre duas máquinas interconectadas via LAN Gigabit no laboratório LARSID/UEFS, permitindo avaliar a latência de *gossip* e o comportamento de sincronização sob rede física real.

4.2. Resultados Funcionais

Os testes de comportamento funcional validaram as rotinas de *failover*, eleição de líder e redundância de recepção de telemetria. A Tabela 2 sumariza os principais cenários testados e as respostas correspondentes do sistema.

Tabela 2. Resultados funcionais do sistema diante de falhas injetadas. Fonte: Próprio Autor.

ID	Cenário de Falha	Resposta do Sistema	Status
TR-01	Queda do <i>Broker</i> B3 (Nordeste)	B2 assume o Setor Nordeste via <i>Ring Failover</i> ; drones executam <i>fall-back</i>	OK
TR-02	Emissão de Alerta <i>Multi-cast</i>	Recebido de forma redundante por todos os <i>brokers</i> ativos	OK
TR-03	Drone Abatido em Missão	<i>Broker</i> detecta perda de <i>socket</i> TCP, remove drone e re-enfileira ocorrência	OK
TR-04	Conflito de <i>Timestamps</i>	<i>Brokers</i> ordenam ocorrências por prioridade e <i>timestamp</i> de Lamport	OK
TR-05	Queda do Líder B9 (Centro)	Seguidores detectam ausência de <i>heartbeat</i> e realizam eleição Bully	OK

4.3. Testes Automatizados

Testes unitários validaram a ordenação e o *aging* do *heap* de prioridades com inserção concorrente de chaves duplicadas. Testes de integração verificaram o *pipeline* de serialização JSON em *sockets* TCP/IP e a consistência dos relógios de Lamport sob carga massiva de *goroutines*.

4.4. Carga e Desempenho

As métricas de latência e consumo de recursos foram coletadas durante uma sessão de estresse de 10 minutos, com geração de alertas na taxa de 1 evento a cada 2 segundos por sensor. A Tabela 3 consolida os resultados quantitativos observados.

Tabela 3. Métricas de desempenho obtidas em teste de estresse contínuo. Fonte: Próprio Autor.

Métrica	Valor	Observação
Tempo de despacho médio (s)	1,5	Da ocorrência ao despacho do drone
Divergência de relógios de Lamport (ms)	0	Filas idênticas entre <i>brokers</i>
Uso médio de CPU por <i>broker</i> (%)	< 5%	Concorrência via <i>goroutines</i>
Uso médio de RAM por <i>broker</i> (MB)	12	Inclui <i>overhead</i> do <i>heap</i> binário
Taxa de sucesso de missões (%)	≈ 90%	Taxa fixa de 10% de abates simulados
Latência de comunicação P2P (ms)	< 10	Média nas conexões da malha TCP/IP

4.5. Comparação com FarmNode

A Tabela 4 estabelece uma comparação estrutural e funcional direta entre o FarmNode e o HormuzNet, evidenciando os avanços de robustez obtidos.

Tabela 4. Comparação entre FarmNode (centralizado) e HormuzNet (descentralizado). Fonte: Próprio Autor.

Critério	FarmNode (P1)	HormuzNet (P2)
Topologia	<i>Broker</i> único centralizado	Malha de N <i>brokers</i>
Ponto único de falha	Existe	Eliminado (<i>fallback</i> + <i>heartbeat</i>)
Coordenação de recursos	Implícita (1 servidor decide)	Distribuída (cascata + fila local)
Fila de requisições	Local no <i>broker</i>	Local + sincronização por <i>broadcast</i>
Protocolo <i>broker</i> ↔ <i>broker</i>	Não existe	TCP/IP com JSON por linha
Deteção de falha	<i>Timeout</i> de inatividade	<i>Heartbeat</i> TCP 5 s, <i>timeout</i> 15 s
Critério de despacho	Bateria disponível	Proximidade cartesiana
Dispositivos simulados	Sensores ambientais	Sensores marítimos + drones

4.6. Discussão e Limitações

Os dados confirmam que o HormuzNet elimina pontos únicos de falha: a inoperância de *brokers* de setor não interrompe o monitoramento tático, pois o *Ring Failover* delega a responsabilidade geográfica ao nó herdeiro no anel lógico enquanto os drones migram via *fallback* TCP/IP em segundos. A sincronização de Lamport impede despacho duplicado mesmo sob partição temporária de rede. Como principais limitações, destacam-se o crescimento quadrático do tráfego de *broadcast* de filas com o número de *brokers* e a dependência de roteamento IGMP para UDP *Multicast*, que pode limitar a implantação em redes WAN sem tunelamento.

5. Conclusão

O desenvolvimento do HormuzNet demonstrou a viabilidade de aplicar conceitos clássicos de sistemas distribuídos na construção de soluções de monitoramento marítimo tolerantes a falhas e de alta disponibilidade. A transição de uma topologia com *broker* centralizado (FarmNode) para uma malha descentralizada e dinâmica eliminou com eficácia os pontos únicos de falha, garantindo a continuidade do monitoramento mesmo em cenários de queda em cascata de nós.

As decisões de projeto fundamentadas na teoria — telemetria redundante via UDP/IP *Multicast* [Postel 1980], ordenação distribuída por Relógios Lógicos de Lamport [Lamport 1978] e eleição de coordenador via Algoritmo Bully [Tanenbaum et al. 2021] — foram validadas sob injeção contínua de falhas e estresse de carga. O sistema alcançou auto-recuperação automática de *brokers* e contornou com eficiência a concorrência pelo despacho de drones, mantendo filas de ocorrências íntegras e consistentes em toda a malha. O modelo de concorrência de Go com *goroutines* mostrou-se especialmente adequado para a paralelização massiva exigida por dezenas de conexões simultâneas.

Como trabalhos futuros, destacam-se: a integração de canais TLS na malha P2P para proteção das mensagens de coordenação; o uso de protocolos de roteamento dinâmico para viabilizar implantações em redes WAN, eliminando a dependência de IGMP do *Multicast*; e a adoção do algoritmo Raft em substituição ao Bully para maior tolerância a partições de rede. Em suma, o HormuzNet estabelece uma base funcional e robusta para sistemas de monitoramento autônomos em ambientes operacionais críticos.

Referências

- Deering, S. (1989). Host extensions for ip multicasting. RFC 1112, RFC Editor.
- Donovan, A. A. and Kernighan, B. W. (2015). *The Go Programming Language*. Addison-Wesley Professional.
- Fette, I. and Melnikov, A. (2011). The websocket protocol. RFC 6455, RFC Editor.
- Institute, I. S. (1981). Transmission control protocol. RFC 793, RFC Editor.
- Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565.
- Postel, J. (1980). User datagram protocol. RFC 768, RFC Editor.
- Postel, J. (1981). Internet protocol. RFC 791, RFC Editor.

Tanenbaum, A. S., Wetherall, D. J., and Feamster, N. (2021). *Redes de Computadores*. Pearson.